Part II: Hyperparameter Transfer & Scaling MoEs

Collaborators



Moritz Haas



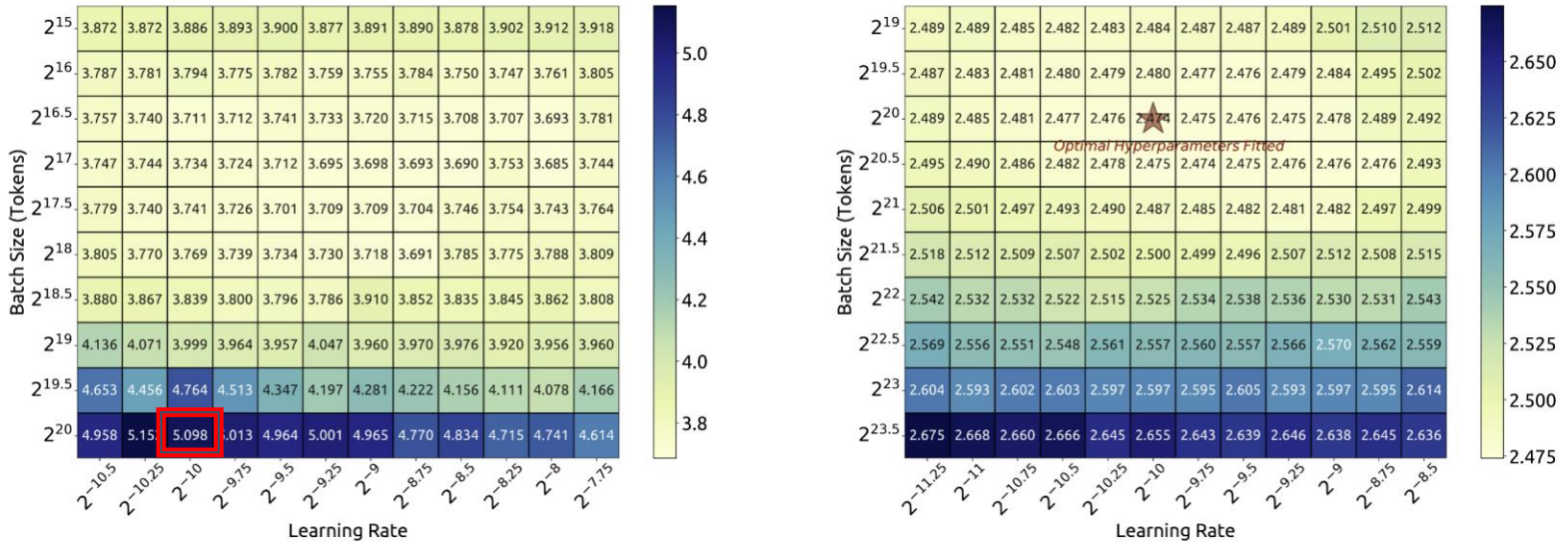
Leena C Vankadara

Topics Covered

- What is hyperparameter transfer and why is it important?
- A very brief into to scaling theory and the maximal update parameterization
- The importance of measuring what happens during neural network training
- A practical lesson from scaling MoEs

Motivation

Optimal neural network hyperparameters usually change with scale. For example, the optimal learning rate and batch size of AdamW depend on the model size and training duration.



(a) 1e17 FLOPs (177M FLOPs/token)

(b) 1e20 FLOPs (2.94B FLOPs/token)

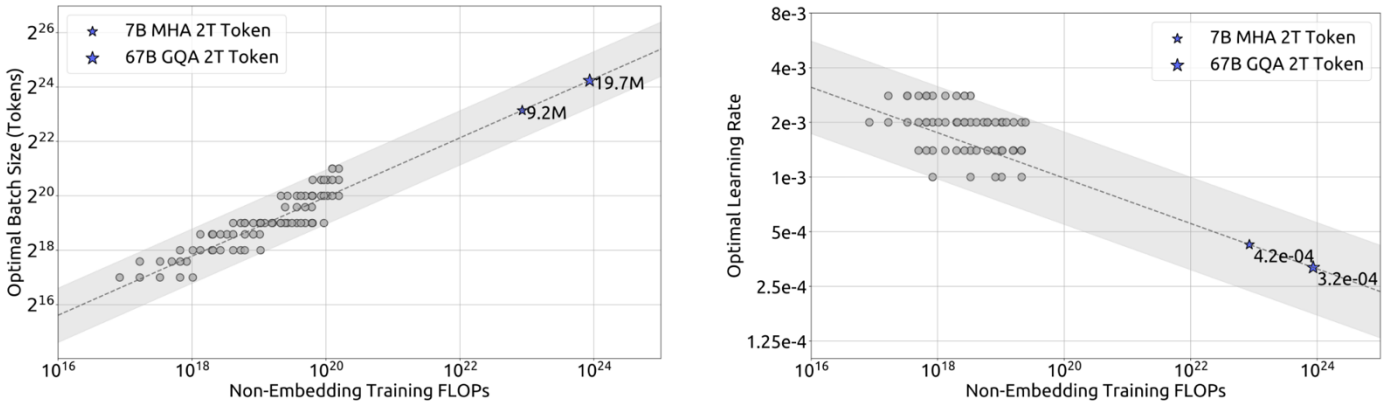
Figure 2 | Training loss w.r.t. batch size and learning rate with 1e17 and 1e20 FLOPs.

DeepSeek-AI, *DeepSeek LLM: Scaling Open-Source Language Models with Longtermism*, 2024

Solution: Scaling Laws for Hyperparameters

near-optimal parameters within this interval. The final formulae we fitted for batch size and learning rate are as follows:

$$\eta_{\text{opt}} = 0.3118 \cdot C^{-0.1250}$$
$$B_{\text{opt}} = 0.2920 \cdot C^{0.3271} \quad (1)$$



(a) Batch size scaling curve

(b) Learning rate scaling curve

Figure 3 | Scaling curves of batch size and learning rate. The grey circles represent models whose generalization error exceeded the minimum by no more than 0.25%. The dotted line represents the power law fitting the smaller model. The blue stars represent DeepSeek LLM 7B and 67B.

DeepSeek-AI, *DeepSeek LLM: Scaling Open-Source Language Models with Longtermism*, 2024

Can we design training recipes that are ready to be scaled?

Needs Scaling Law [1]

Hyperparameter Transfer

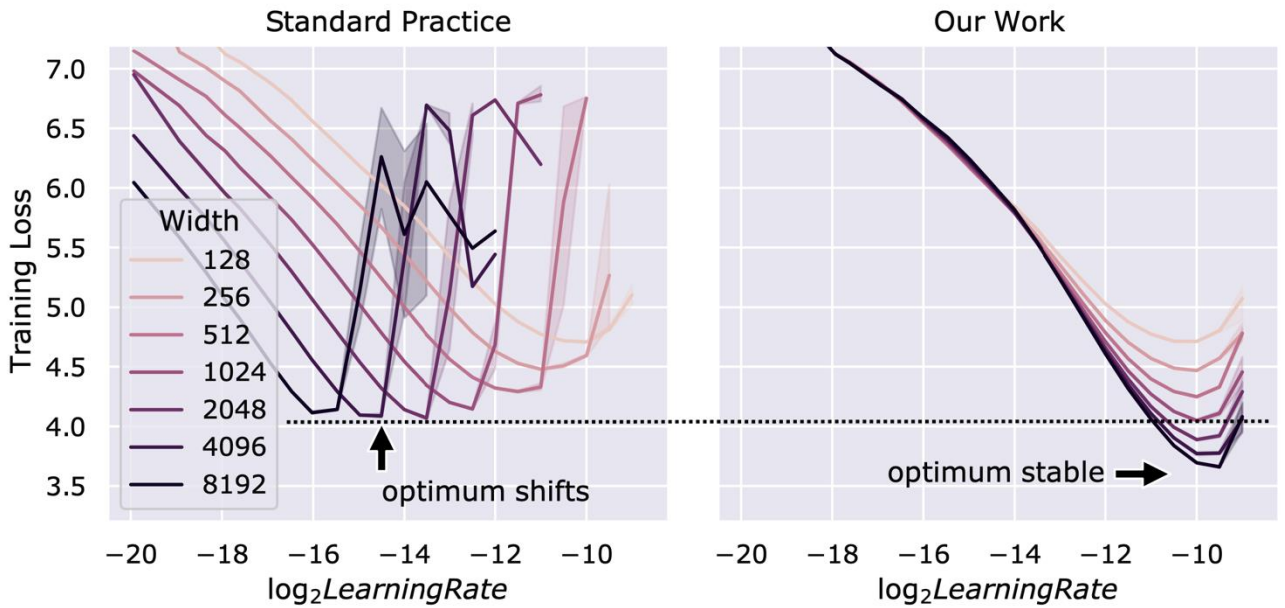
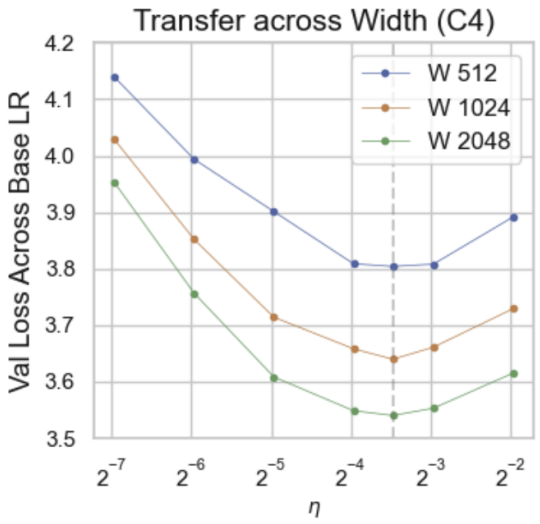


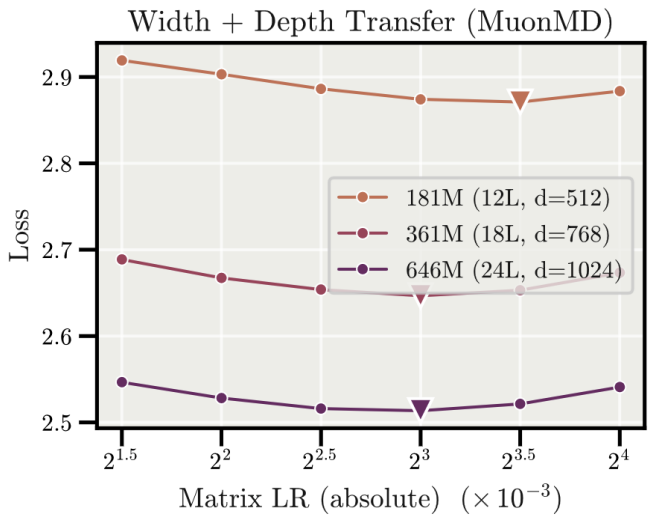
Figure 1: Training loss against learning rate on Transformers of varying d_{model} trained with Adam.

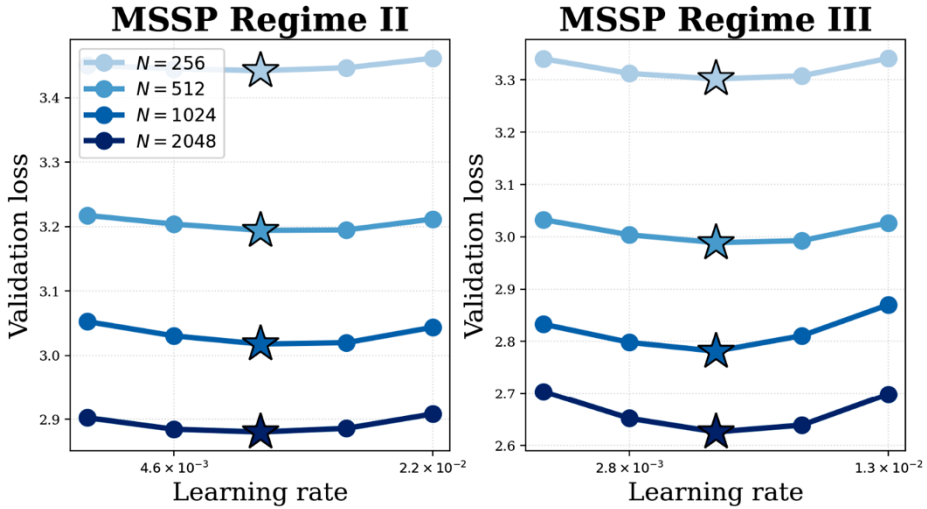
[1] That a reliable scaling law exists is not guaranteed! (phase transitions)

Yang et al, *Tensor Programs V: Tuning Large Neural Networks via Zero-Shot Hyperparameter Transfer*, 2021
Haas et al., *On the Surprising Effectiveness of Large Learning Rates under Standard Width Scaling*, 2025

Hyperparameter Transfer: Examples







Vankadara et al. *How to Scale Mixture-of-Experts: From μP to the Maximally Scale-Stable Parameterization*, 2026
Hägele et al., *Improving Neural Network Training by Decoupling the Magnitude and Direction of Weight Vectors*, 2026
Jiang et al. *Hyperparameter transfer for Mixture-of-Experts*, 2026

When do we have hyperparameter transfer?

Intuitively, we have hyperparameter transfer when we have **stable scaling.**

Informally, stable scaling means that the training dynamics do not change as we scale up the neural network.

In practice, a lot of things change as we scale up a neural network!

With standard methods, small-scale training is usually stable, but the more we scale up, the more unstable the training dynamics become (loss spikes, gradient norm growth, convergence to sub-optimal solutions).

Can we understand how the training dynamics change as we scale up a neural network?

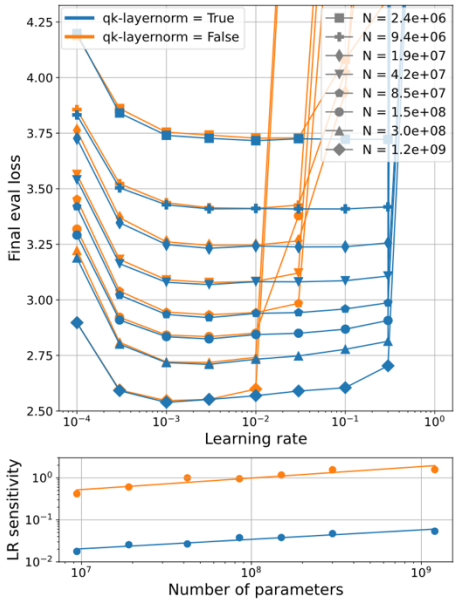
Throw-in : A nice paper about training stability

Small-scale proxies for large-scale Transformer training instabilities

Mitchell Wortsman Peter J. Liu Lechao Xiao Katie Everett
Alex Alemi Ben Adlam John D. Co-Reyes Izzeddin Gur Abhishek Kumar
Roman Novak Jeffrey Pennington Jascha Sohl-dickstein Kelvin Xu
Jaehoon Lee* Justin Gilmer* Simon Kornblith*
Google DeepMind

Abstract

Teams that have trained large Transformer-based models have reported training instabilities at large scale that did not appear when training with the same hyperparameters at smaller scales. Although the causes of such instabilities are of scientific interest, the amount of resources required to reproduce them has made investigation difficult. In this work, we seek ways to reproduce and study training stability and instability at smaller scales. First, we focus on two sources of training instability described in previous work: the growth of logits in attention layers (Dehghani et al., 2023) and divergence of the output logits from the log probabilities (Chowdhery et al., 2022). By measuring the relationship between learning rate and loss across scales, we show that these instabilities also appear in small models when training at high learning rates, and that mitigations previously employed at large scales are equally effective in this regime. This prompts us to investigate the extent to which other known optimizer and model interventions influence the sensitivity of the final loss to changes in the learning rate. To this end, we study methods such as warm-up, weight decay, and the μParam (Yang et al., 2022), and combine techniques to train small models that achieve similar losses across orders



Wortsman et al. *Small-scale proxies for large-scale Transformer training instabilities* 2023

Obtaining stable scaling is difficult in practice

Most publicly released LLM training recipes (Deepseek, Kimi, Qwen, OLMo, Marin, …) scale sufficiently stable in practice (because they can train good models), but (presumably) their scaling is not stable enough to get hyperparameter transfer (and to scale further? 🤔).

In LLM technical reports, we often see “stability hacks” that aim to fix scaling issues.

- Deepseek V4: Address stability issues in MOE layers by occasionally using historical routing parameters
- Kimi K3: Use QB expert biases and apply Muon separately for every attention head
- Gemma 2: Logit soft-capping via tanh
- Rewind training and skip batches that cause loss and gradient spikes, use z-loss on diverging logits, add normalization layers where you have problems …

Width-scaling with Adam and μP

- Assume that we have a stable training recipe for a small model. Now we would like to scale the width n of the model. What happens and can we achieve stable scaling?
- Consider what happens to the activations of dense layers in the neural network:
  - One unit of the next layer: $h_i = \sum_{j=1}^{n} W_{ij}\ x_j$
  - At initialization, $|\ h_i\ | \approx \sqrt{n}\,\sigma$ (CLT-like scaling of the term because of independence)
  - With $\sigma \approx 1/\sqrt{n}$ (He/LeCun init), we get $|\ h_i\ | = \Theta(1)$

Takeway: The Initialization stabilizes the forward pass — at step 0.

Width-scaling with Adam and μP

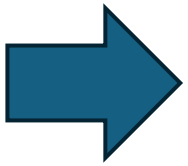
- Now we take a gradient step with Adam and ask how this effects the next forward pass.
- Adam performs coordinate-wise normalization: Let us heavily simplify this and assume that every coordinate moves by $\approx \eta$, regardless of gradient magnitude (Sign-descent)
- Effect on the next forward pass: $\Delta h_i = \sum_j \Delta W_{ij}\, x_j \approx \pm\, \eta \sum_j |\, x_j\, | = \Theta(\eta \cdot n)$.
- The key point in this derivation is that the terms in the sum are now correlated: This is because the update ΔW_{ij} is built from x
- We want $\Delta h_i = \Theta(\eta \cdot n) = \Theta(1)$: This can be achieved by multiplying η with $1/n$

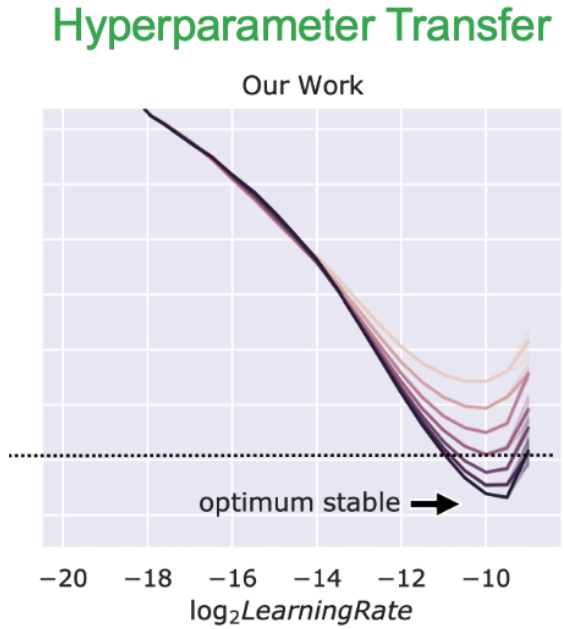
Takeway: To stabilize the forward passes at steps>0, we need layer-wise learning rates.

Width-scaling with Adam and μP

This leads to the maximal update parameterization (μP).

tensor	init std	Adam LR
embedding / input, biases, LN	$\Theta(1)$	$\Theta(1)$
hidden matrices (fan-in n)	$1/\sqrt{n}$	η/n
readout	$\sim 1/n$ (or $1/\sqrt{n}$ + output multiplier)	η/n





Yang & Littwin (2023) show these scalings give the (essentially unique) learning in the infinite-width limit that has $\Theta(1)$ for weights, activations and gradients.

Modern Extensions

Don't be lazy: CompleteP enables compute-efficient deep transformers

Nolan Dey* Cerebras Systems
Bin Claire Zhang* Cerebras Systems
Lorenzo Noci ETH Zurich, Princeton University
Mufan Li Princeton University
Blake Bordelon Harvard University
Shane Bergsma Cerebras Systems
Cengiz Pehlevan Harvard University, Kempner Institute
Boris Hanin Princeton University
Joel Hestness Cerebras Systems

Abstract

We study compute efficiency of LLM training when using different *parameterizations*, i.e., rules for adjusting model and optimizer hyperparameters (HPs) as model size changes. Some parameterizations fail to *transfer* optimal base HPs (such as learning rate) across changes in model depth, requiring practitioners to either re-tune these HPs as they scale up (expensive), or accept sub-optimal training when re-tuning is prohibitive. Even when they achieve HP transfer, we develop theory to show parameterizations may still exist in the *lazy learning* regime where layers learn only features close to their linearization, preventing effective use of depth and nonlinearity. Finally, we identify and adopt the parameterization we call *CompleteP* that achieves both depth-wise HP transfer and non-lazy learning in all layers. CompleteP enables a wider range of model width/depth ratios to remain compute-efficient, unlocking shapes better suited for different hardware settings and operational contexts. Moreover, CompleteP enables 12-34% compute efficiency improvements over the prior state-of-the-art. All experiments were run on Cerebras CS-3 systems. A minimal implementation is available at https://github.com/EleutherAI/nanoGPT-mup/tree/completep.

Scaling across width and depth

How to Scale Mixture-of-Experts: From μP to the Maximally Scale-Stable Parameterization

Leena Chennuru Vankadara[1,*] **Moritz Haas**[2,*]
Luke Hayward[1,†] **Sebastian Bordt**[3,†] **Alessandro Breccia**[1,†]
[1]Gatsby Computational Neuroscience Unit, University College London
[2]Amazon [3]University of Tübingen, Tübingen AI Center

Abstract

Recent frontier large language models predominantly rely on Mixture-of-Experts (MoE) architectures. Despite empirical progress, there is still no principled understanding of how hyperparameters should scale with network width N, expert width N_e, number of experts M, sparsity K, and depth L to ensure both stability and optimal performance at scale. We take a principled step toward resolving this gap by analyzing three different scaling regimes: (I) co-scaling $N \asymp N_e$, (II) co-scaling $N \asymp M \asymp K$, and (III) full proportional scaling of N, N_e, M, and K. For each regime, we develop a novel Dynamical Mean Field Theory (DMFT) description of the limiting training dynamics of MoEs that provides a formal foundation for our analysis. Within this framework, we derive the unique parameterization for SGD and Adam satisfying all maximal-update (μ) desiderata. We then show that the resulting μP prescription does not reliably induce monotonic improvement with scale or robust learning-rate transfer. We trace these pathologies to scale-dependent observables in the aggregation dynamics, which motivates a refined set of desiderata that we term *maximal scale stability*. Guided by this principle, we derive a *Maximally Scale-Stable Parameterization* (MSSP) for both SGD and Adam in all three scaling regimes, and characterize the corresponding limiting dynamics - qualitatively distinct from the μP limit - through a separate DMFT analysis. Experiments verify that MSSP robustly recovers learning rate transfer and monotonic improvement with scale across regimes. Combined with existing depth-scaling theory, these results provide a complete scaling prescription for MoE architectures as a function of width, depth, expert width, and number of experts.

Scaling MoEs

Training Deep Learning Models with Norm-Constrained LMOs

Thomas Pethick [1] **Wanyun Xie** [1] **Kimon Antonakopoulos** [1]
Zhenyu Zhu [1] **Antonio Silveti-Falls** [2] **Volkan Cevher** [1]

Abstract

In this work, we study optimization methods that leverage the linear minimization oracle (lmo) over a norm-ball. We propose a new stochastic family of algorithms that uses the lmo to adapt to the geometry of the problem and, perhaps surprisingly, show that they can be applied to unconstrained problems. The resulting update rule unifies several existing optimization methods under a single framework. Furthermore, we propose an explicit choice of norm for deep architectures, which, as a side benefit, leads to the transferability of hyperparameters across model sizes. Experimentally, we demonstrate significant speedups on nanoGPT training using our algorithm, Scion, without any reliance on Adam. The proposed method is memory-efficient, requiring only one set of model weights and one set of gradients, which can be stored in half-precision. The code is available at https://github.com/LIONS-EPFL/scion.

Adaptation on-the-fly has been the defacto standard in this setting, with adaptive algorithms, such as Adam (Kingma, 2014), dominating the deep learning model training.

One possible way for adaptation a priori, which we focus on in this work, is to modify the underlying norm used to measure distances in the parameter space. There is precedence to our proposal, as the early work by Carlson et al. (2015a;b;c) introduced the stochastic spectral descent method (SSD), which performs steepest descent in the spectral norm, and demonstrated that the method can substantially accelerate deep learning training.

The significance of the SSD approach has been very recently brought back to attention by Bernstein & Newhouse (2024b), who showed that the Shampoo optimizer (Gupta et al., 2017)—winner of the external tuning track at the 2024 AlgoPerf: Training Algorithms competition (Dahl et al., 2023)—can be viewed as SSD when a certain accumulation is disabled. Moreover, Bernstein & Newhouse (2024b) introduced an efficient Newton-Schultz iteration to replace the approximate SVD calculations previously required. Jordan et al. (2024b) incorporated the Newton-

Scaling with Muon

Scaling Theory: Limitations

- Modern scaling theory can describe a fair amount of scaling limits of neural networks (width scaling, depth scaling, scaling MOE's)

- However, some scaling limits like training duration, batch size, and depth are not fully resolved (and people disagree about this ☺)

- Moreover, the theory can only describe certain limits, and not everything that you might want to do in practice falls neatly into the settings that can be described by the theory

- It is not 100% understood under what conditions hyperparameter transfer arises. In practice, we still need to tune layer-wise multipliers at base size for clean transfer (and we don't know why).

Coordinate Checks

Measure how activations over the course of training depend on the scaling factor

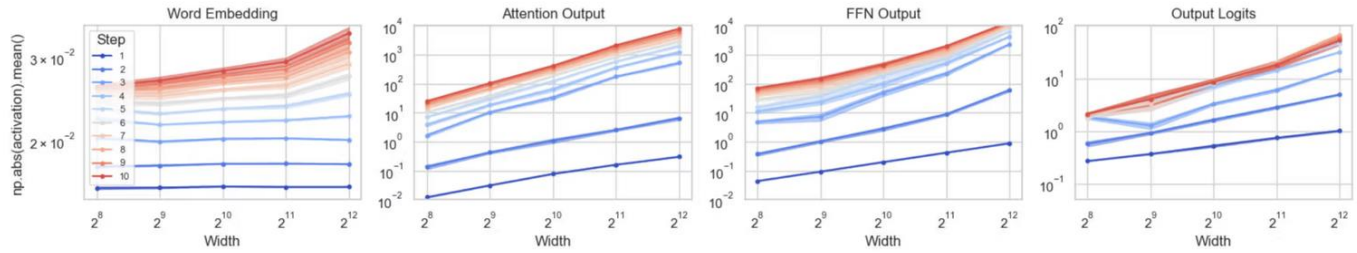


Figure 5: Coordinate check for SP.

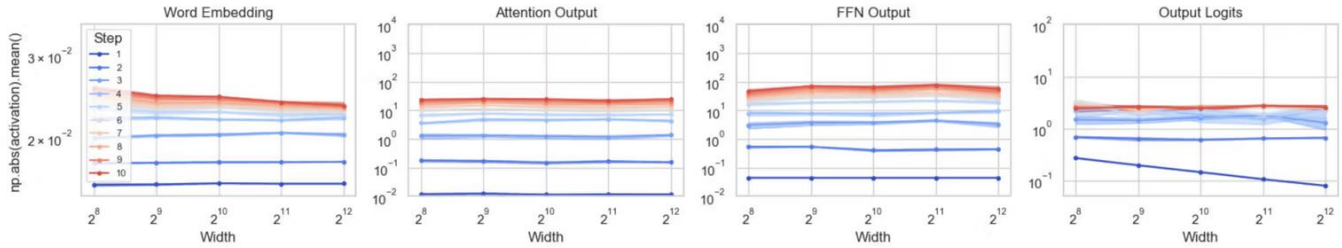
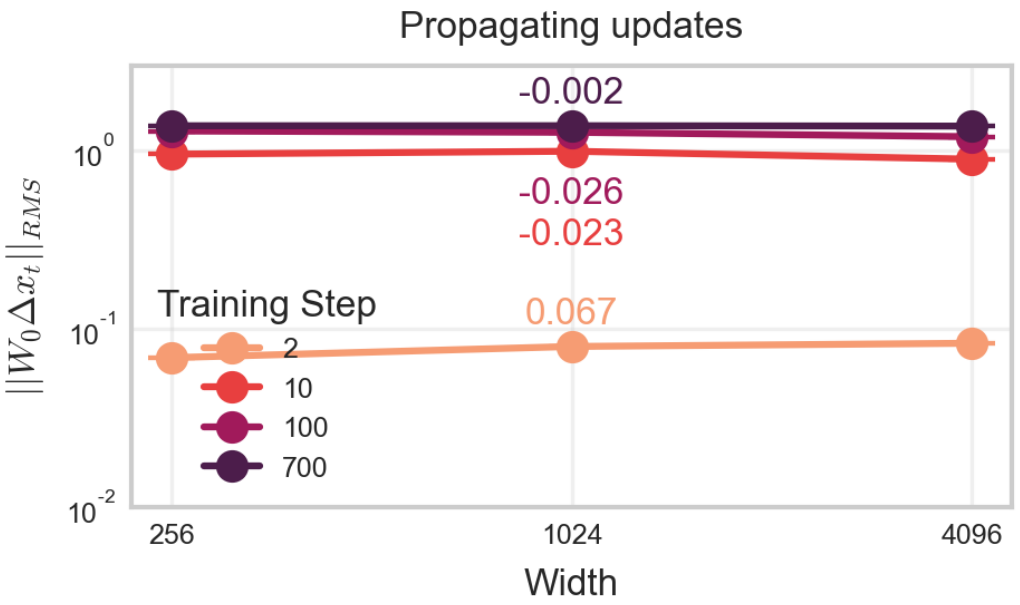


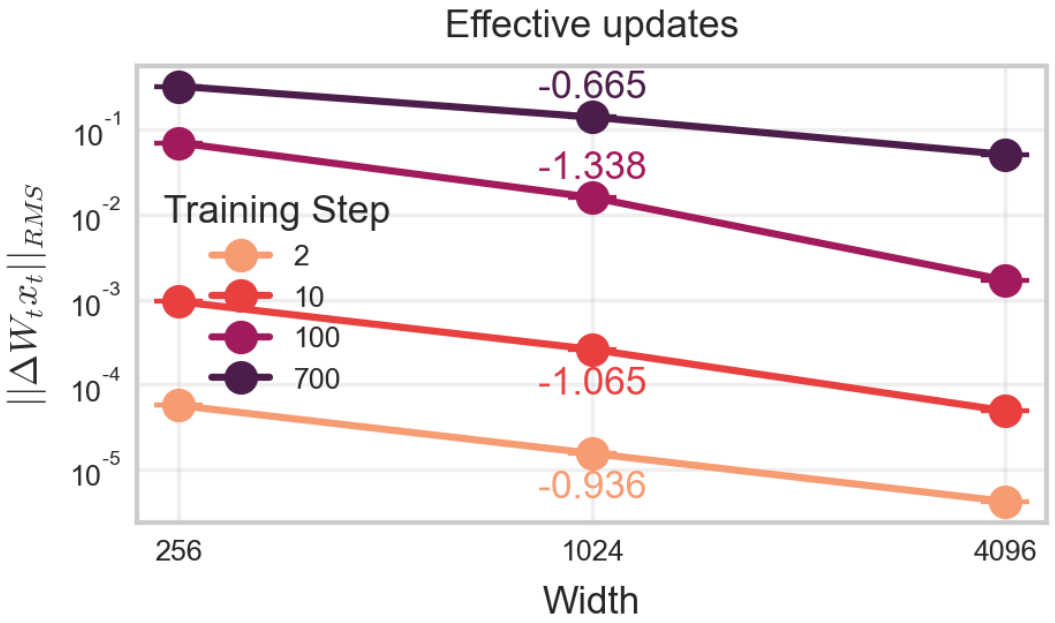
Figure 10: Coordinate check for μP

https://www.cerebras.ai/blog/the-practitioners-guide-to-the-maximal-update-parameterization

Refined Coordinate Checks

Measure how **changes** to activations and weights depend on the scaling factor

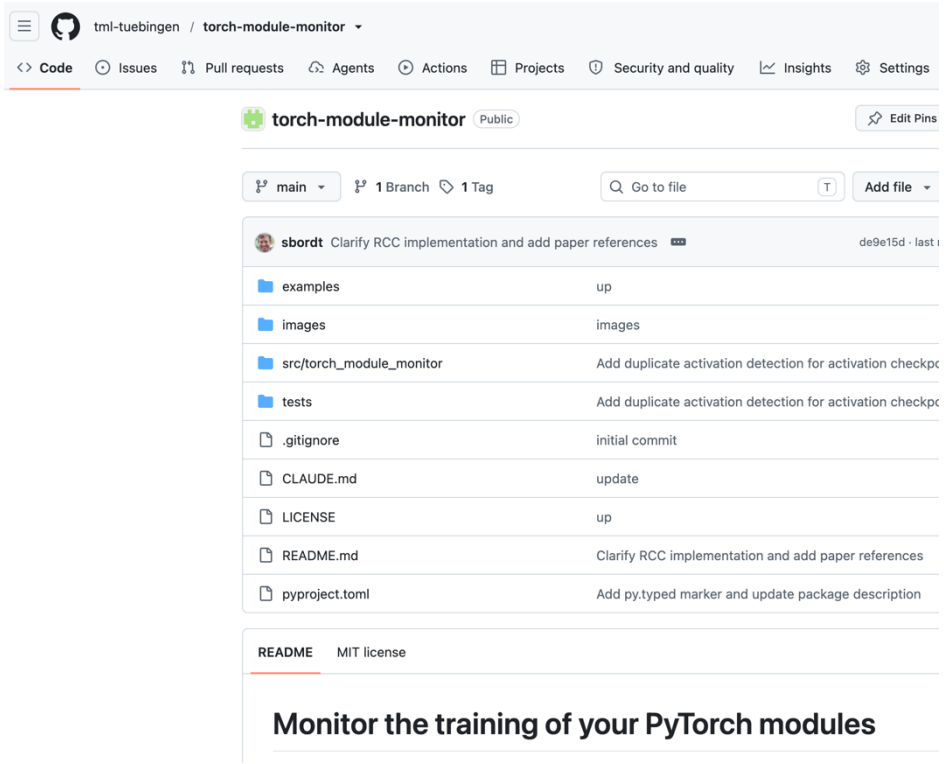


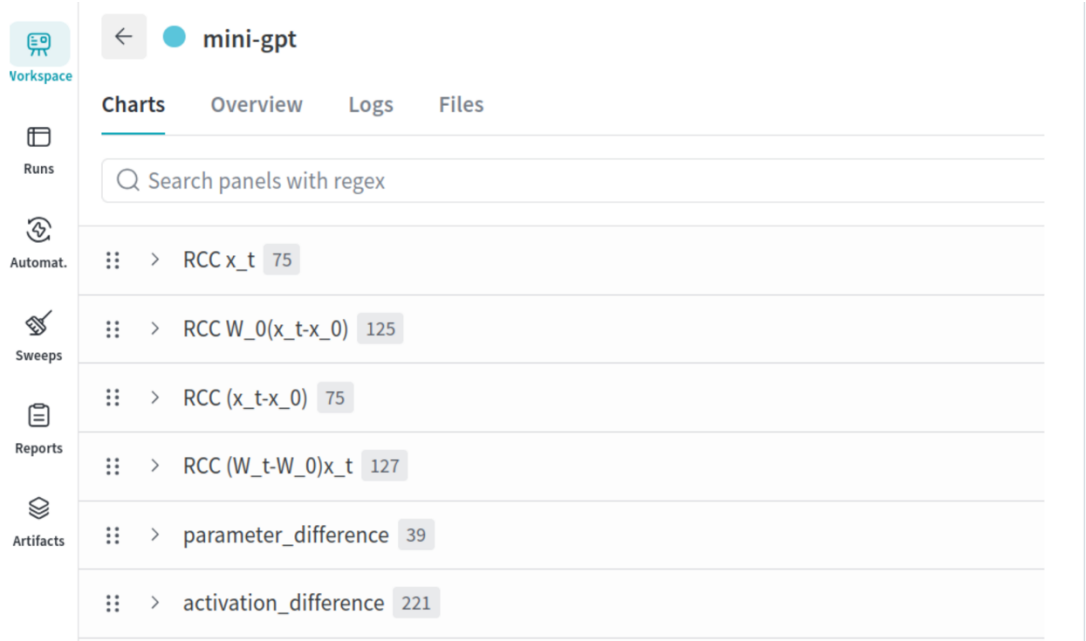


https://sbordt.substack.com/p/how-to-measure-whether-your-training

Refined Coordinate Checks in PyTorch

Measure how **changes** to activations and weights depend on the scaling factor





https://github.com/tml-tuebingen/torch-module-monitor

A Practical Lesson from Scaling MoEs

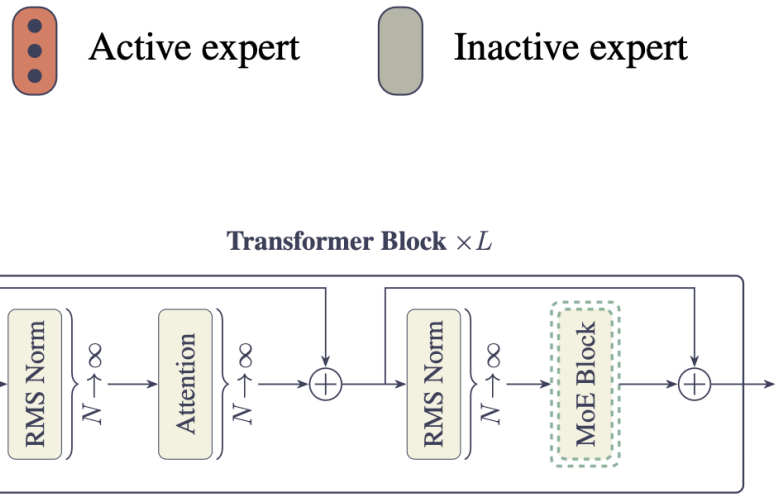
Can we have stable scaling with Mixture-of-Experts models (MoEs)?

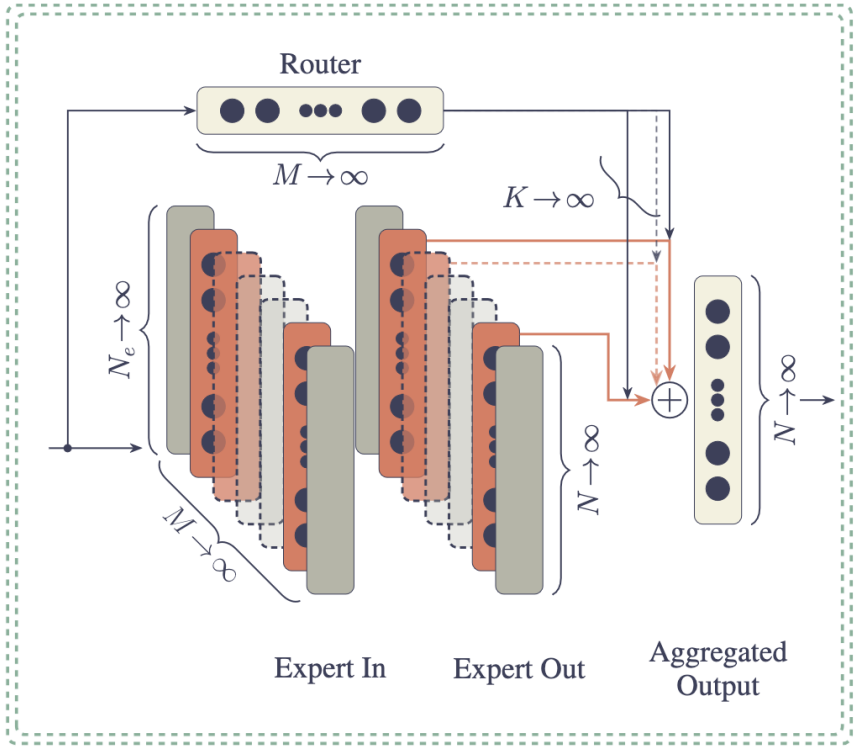
- MoEs have several different scaling limits!
- If we fix the number of experts and just scale the width / depth, then we have “standard” scaling limits
- However, it gets more complicated when the number of experts increases with the size of the model
- Let us consider a scaling regime where the number of experts scales proportional the width of the model (and the width of the experts scales as well)

Vankadara et al. *How to Scale Mixture-of-Experts: From μP to the Maximally Scale-Stable Parameterization*, 2026

A Practical Lesson from Scaling MoEs

The number of experts and expert width increase with the size of the model.





(d) Regime III: $N \asymp N_e \asymp M \asymp K \to \infty$.

Vankadara et al. *How to Scale Mixture-of-Experts: From μP to the Maximally Scale-Stable Parameterization*, 2026

A Practical Lesson from Scaling MoEs

MoEs always have some kind of auxiliary load-balancing strategy.

- For example, a loss that encourages expert balance.
- A popular form of the aux loss was introduced in Fedus et al. (2022)

$$\mathcal{L}_{\text{aux}} = \alpha \cdot N \sum_{i=1}^{N} f_i \, P_i$$

Alpha = tunable hyperparameter
N = Number of Experts,
f_i =fraction of tokens seen by expert i
P_i = the mean routing probability of expert i

Fedus, Zoph & Shazeer, *"Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity"* (JMLR 2022, arXiv 2101.03961)

A Practical Lesson from Scaling MoEs

MoEs always have some kind of auxiliary load-balancing strategy.

- For example, a loss that encourages expert balance.
- A popular form of the aux loss was introduced in Fedus et al. (2022)

$$\mathcal{L}_{\text{aux}} = \alpha \cdot N \sum_{i=1}^{N} f_i \, P_i$$

Alpha = tunable hyperparameter
N = Number of Experts,
f_i =fraction of tokens seen by expert i
P_i = the mean routing probability of expert i

We initially used this for our MoE scaling experiments, and this was a major problem – why?

Fedus, Zoph & Shazeer, *"Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity"* (JMLR 2022, arXiv 2101.03961)

A Practical Lesson from Scaling MoEs

MoEs always have some kind of auxiliary load-balancing strategy.

- For example, a loss that encourages expert balance.
- A popular form of the aux loss was introduced in Fedus et al. (2022)

$$\mathcal{L}_{\text{aux}} = \alpha \cdot N \sum_{i=1}^{N} f_i \, P_i$$

Alpha = tunable hyperparameter
N = Number of Experts,
f_i =fraction of tokens seen by expert i
P_i = the mean routing probability of expert i

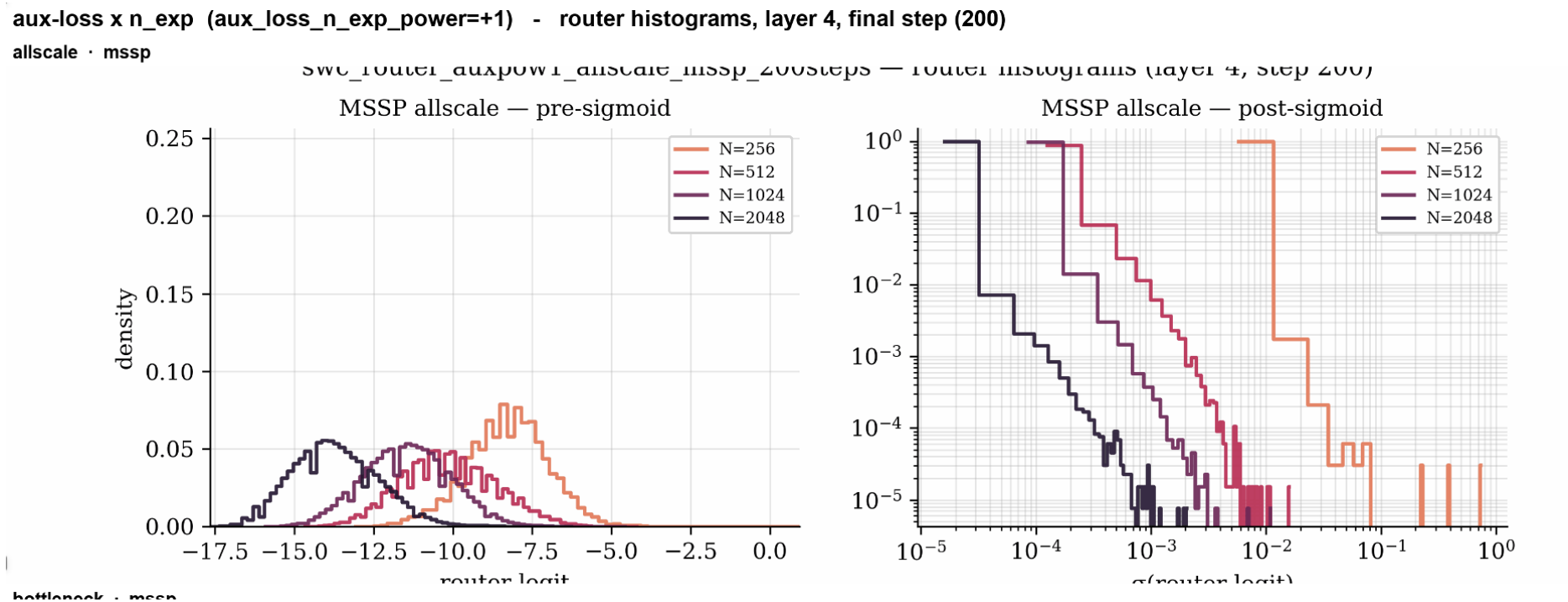
We initially used this for our MoE scaling experiments, and this was a major problem – why?

The magnitude of the loss scales TWICE with the numer of expert!

Fedus, Zoph & Shazeer, *"Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity"* (JMLR 2022, arXiv 2101.03961)

A Practical Lesson from Scaling MoEs

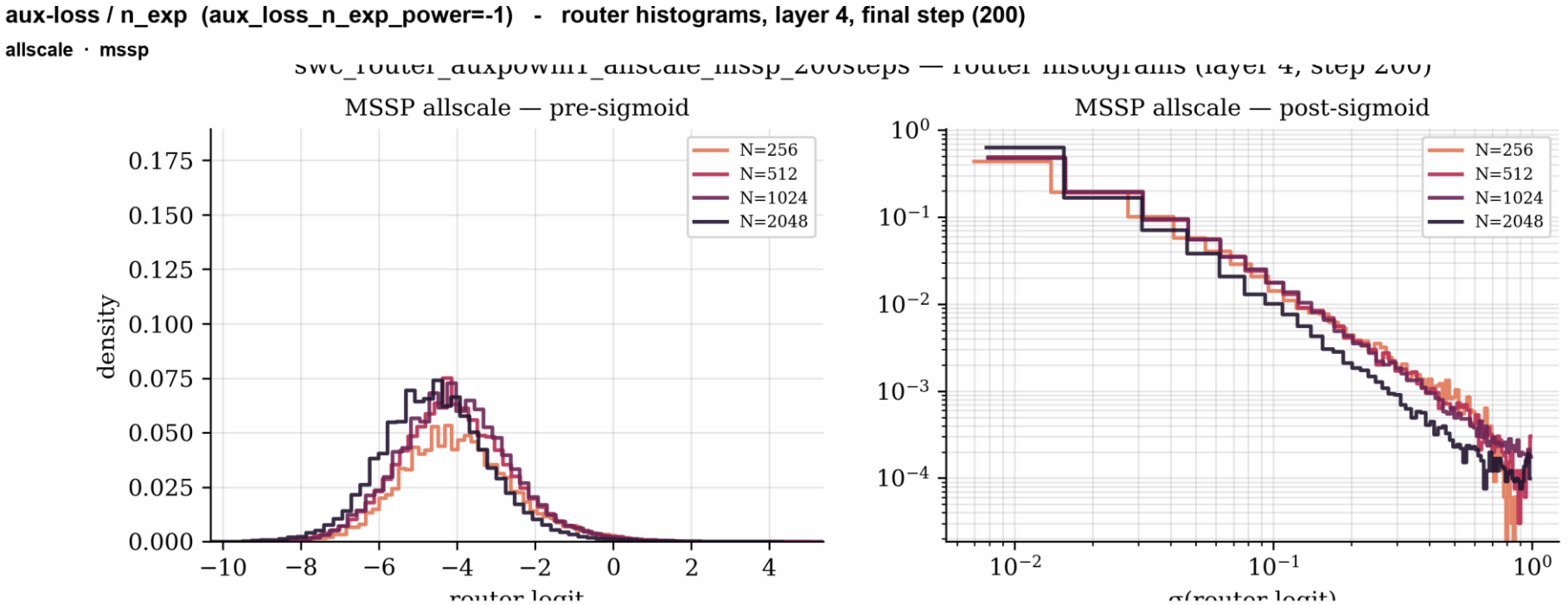
Let us consider how the histogram of router logits and activations changes with scale (sigmoid-routing)



Aux-loss from Fedus et al. (2022)

A Practical Lesson from Scaling MoEs

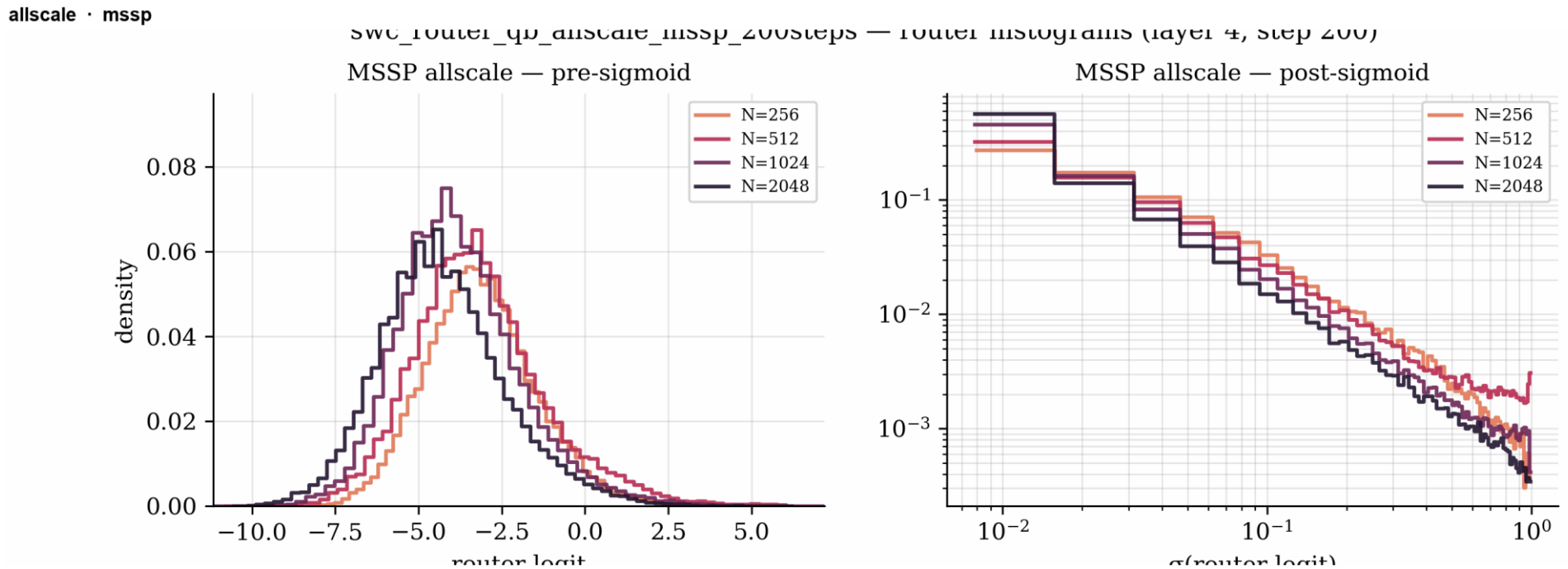
Let us consider how the histogram of router logits and activations changes with scale (sigmoid-routing)



Normalizing the sum with N^-1 instead of N

A Practical Lesson from Scaling MoEs

Let us consider how the histogram of router logits and activations changes with scale (sigmoid-routing)



QB Expert biases (KimiK3)